ASSIGNMENT CLA-02

Modern Control Systems (EEE311)

Compensators In Matlab And Simulink

Submitted by

Kamineni Dhana Sri Keerthi

AP24110040009
Department of Electrical and Electronics Engineering
SRM University AP

Submitted to

Dr Sumanta Kumar Nanda

Assistant Professor
Department of Electrical and Electronics Engineering
SRM University AP



SRM University
Andhra Pradesh

Department of Electrical and Electronics Engineering
School of Engineering and Sciences (SEAS).

October 2026

24/10/26
Saturday

Modern Control Systems
EEE - 311

K. Dhana sri keerthi
AP24110040009

ASSIGNMENT

Q1. Solve one lead compensator & lag compensator numerical using root locus & bode plot.

Part A: Lead compensator - root locus

Given: $G_p(s) = \frac{1}{s(s+2)(s+10)}$

$M_p \leq 10\% = 0.1$

$T_s \leq 2$ sec

step 01: desired closed loop pole s_d

$\Rightarrow s_d = -\zeta\omega_n \pm j\omega_d$

& $\omega_d = \omega_n\sqrt{1-\zeta^2}$

01. Find ζ from M_p

$M_p = e^{\frac{-\zeta\pi}{\sqrt{1-\zeta^2}}}$

$\Rightarrow \log_e[0.1] = \frac{-\zeta\pi}{\sqrt{1-\zeta^2}}$

$\Rightarrow -2.30258 = \frac{-\zeta\pi}{\sqrt{1-\zeta^2}}$

$\Rightarrow [-2.30258]^2 = \frac{\zeta^2\pi^2}{1-\zeta^2}$

$\Rightarrow \zeta^2 = 0.379$

$\Rightarrow \boxed{\zeta = 0.591154}$

02. Find ω_n

$T_s = \frac{4}{\zeta\omega_n}$

$\Rightarrow T_s = \frac{4}{0.591154\,\omega_n}$

$\Rightarrow \omega_n = \frac{2}{0.591154}$

$\boxed{\omega_n = 3.38320 \text{ rad/s}}$

$\Rightarrow \boxed{s_d = -2 \pm j2.7288}$

Step 02: Angle of deficiency

$\Rightarrow \theta = \tan^{-1}[S_d - S_0]$

$\Rightarrow$ Angle $\angle G(S_d) = \Sigma$ Zeroes $- \Sigma$ poles

$\angle G(S_d) = (2m+1)180^\circ$

$S_0 = 0, -2, -10$

$\theta_1 = \tan^{-1}\left[\frac{-2.7288}{2}\right]$

$= 180 - 53.76^\circ$

$\boxed{\theta_1 = 126.24^\circ}$

$\theta_2 = \tan^{-1}\left[\frac{2.728}{0}\right]$

$\boxed{\theta_2 = 90^\circ}$

$\theta_3 = \tan^{-1}\left[\frac{2.7288}{8}\right]$

$\boxed{\theta_3 = 18.834}$

$\Rightarrow \angle G(S_d) = 0 - [126.24 + 90^\circ + 18.834]$

$= -235.0730$

$\Rightarrow$ convert into +ve angle $360^\circ - 235.0730 = 124.93^\circ$

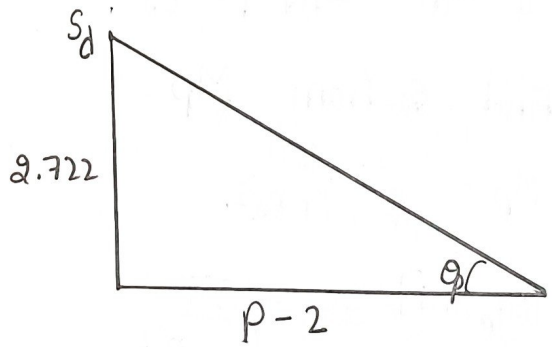
$\Rightarrow$ Take $124.93^\circ \longrightarrow 180^\circ + 55.07^\circ$ compensation.

Step 03: Find pole & zero

$G_c(s) = K_c \frac{(s+z)}{(s+p)} \quad p > z$

$S_d = -2 + j2.7288$

$\Rightarrow$ choose $\boxed{z = -3}$



$\Rightarrow S_d + z = -2 + j2.722 - 3$

$= -5 + j2.722$

$\Rightarrow \tan^{-1}[S_d + z]$ (θ_z) $= \tan^{-1}\left[\frac{2.722}{-5}\right] = 180^\circ - 28.5638 = 151.436^\circ$

$\Rightarrow \theta_z - \theta_p = 55.07^\circ \Rightarrow 151.436^\circ - \theta_p = 55.07^\circ \Rightarrow \theta_p = 96.3661^\circ$

$\Rightarrow \tan[96.3661^\circ] = \frac{2.722}{p-2}$ $\boxed{p = 1.6963}$

$-8.963[p-2] = 2.722$

$-8.963p + 17.926 = 2.722$

$15.20406 = 8.963p$

$p = 1.6963$

$\Rightarrow G_c(s) = K_c \frac{[s-3]}{[s+1.6963]}$

Step 04: Compensated Eq

$$G(S_d) = G_p(S_d)G_c(S_d) = \frac{k_c\,[S_d+3]}{[S_d+1.6]\,S_d\,[S_d+2]\,[S_d+10]}$$

step 05: Find K_c

$\Rightarrow |G(S_d)| = 1$

$$\frac{|k_c|\,|S-3|}{|S+1.6|\,|S|\,|S+2|\,|S+10|} = 1 \Rightarrow$$

$$k_c = \frac{[2.729][3.383][2.72][8.452]}{5.682.}$$

$$S_d = \sqrt{(2)^2+(2.72)^2} = 3.383$$

$$S_d+10 = \sqrt{8^2+2.7^2} = 8.452$$

$$S_d+1.6 = \sqrt{0.16+(2.7)^2} = 2.729.$$

$$S_d+2 = 2.72$$

$$S_d-3 = \sqrt{25+(2.7)^2} = 5.682.$$

$$\boxed{K_c = 36.16924}$$

Step 06: Find equation

$$\boxed{G(S_d) = \frac{36.16924\,[S_d-3]}{[S][S+1.6][S+2][S+10].}}$$

Part B: Lead Compensator - Bode plot

Given: $G(S) = \frac{k}{S[S+4]}$

$e_{ss} \leq 0.05$

$e_s = 0.5$

$T_S = 1.5\ s$

PM = 100

step 01: Find k From e_{ss}

$$K_v = \lim_{S\to 0} S\,(G(S))$$

$$\Rightarrow K_v = \lim_{S\to 0} S\left[\frac{k}{S(S+4)}\right] = K_v = \frac{K}{4} \Rightarrow \frac{k}{4} = \frac{1}{0.05}$$

$$\boxed{k = 80}$$

Step 02: old and new time parameters

$$\Rightarrow TF = \frac{G(S)}{1+G(S)H(S)} = \frac{\frac{80}{S^2+4S}}{1+\frac{80}{S^2+4S}[1]} = \frac{80}{S^2+4S+80}\left[\frac{\omega_n^2}{s^2+2S\omega_n s+\omega_n^2}\right]$$

1. $\omega_n = \sqrt{10} = 8.944$ rad/s
2. $\zeta_0 = \frac{2}{8.944} = 0.223$
3. $T_s = \frac{4}{\zeta_0 \omega_n} = 2$ sec

old

1. $\omega_n = 8.33$ rad/s
2. $\zeta_0 = 0.5$
3. $T_s = 1.5$ sec

new (given)

step 03;

01 $|G(j\omega_{gc})| = 1$

$$\frac{80}{\omega\sqrt{4+\omega^2}} = 1$$

$$= \omega^2[4+\omega^2] = 80$$

$$= \omega^4 + 4\omega^2 - 80^2 = 0$$

$$\Rightarrow x^2 + 4x - 80^2 = 0$$

$$x \begin{cases} \frac{154.7}{2} & \checkmark \\ \frac{-176}{2} & \times \end{cases}$$

$x = 77.39 \Rightarrow \omega = \sqrt{x} = 8.897$

$\boxed{\omega_{gc} = 8.897}$

1. $\alpha = \frac{1+\sin\phi_m}{1-\sin\phi_m}$
2. $\phi_m = PM_{(req)} - PM_{(exist)} + 5°$
3. $PM_{(req)} \rightarrow$ given
4. $PM_{(exist)} = \angle G(j\omega_{gc}) + 180°$
5. $\omega_{gc} = |G(j\omega_{gc})| = 1$

02 $PM_{(exist)} = 180 + \angle G(j\omega_{gc})$

$$\angle G(j\omega_{gc}) = 0° - \tan^{-1}\left[\frac{8.897}{0}\right] - \tan^{-1}\left[\frac{8.897}{4}\right] = -154.82°$$

$PM_{(exist)} \Rightarrow 180° - 154.82° = 25.18°$

03 $PM_{req} = 100[\zeta_{(new)}]$

$\boxed{PM_{req} = 50}$

04 $\alpha = \frac{1+\sin\phi_m}{1-\sin\phi_m}$

$\boxed{\alpha = 2.984}$

step 04:

01. $\omega_{gcn} \Rightarrow \frac{80}{\sqrt{[\omega_{gcn}]^2[\sqrt{(\omega_{gc}]^2+16}]}} = \frac{1}{\sqrt{2.984}}$

$[\omega_{gcn}]^2 \{ \omega_{gcn}^2 + 16 \} = 19090.74$

$\omega_{gcn}^2 = x$

$\Rightarrow x^2 + 16x - 19090.74 = 0$

$x = 130.39$

$\omega_{gcn} = \sqrt{x} = 11.0 \text{ rad/s}$

02. $T = \frac{1}{11[\sqrt{2.984}]} = 0.05075$

01. $G_c(s) = K_c \frac{1+TS}{1+\alpha TS}$

02 $T = \frac{1}{\omega_{gcnew}\sqrt{\alpha}}$

03 $\omega_{gc_{(new)}} \Rightarrow |G(j\omega_{gc_{(new)}}| = \frac{1}{\sqrt{\alpha}}$

ω_{gcn}

03. $G_c(s) = K_c \frac{1+0.0507}{1+[0.15855]}$

step 05: K_c find

$K_c \, |G_c[j\omega_{gcn}]| \, |G_p[j\omega_{gcn}]| = 1$

$K_c \left| \frac{\sqrt{1^2+(0.05)^2}}{\sqrt{1^2+0.158^2}} \right| \left| \frac{80}{\sqrt{(121)^2+(44)^2}} \right| = 1$

$K_c = 2.829$

step 06: Final compensated system.

$G(S) = 2.829 \left[\frac{1+0.0500\,S}{1+0.15855} \right]$

Part C : Lag Compensator - Root locus

Given $G(S) = \frac{K}{S[S+2][S+4]}$

$\zeta = 0.5$

$K_v = 20$

$T_S = 2\text{sec}$

step 01: Find the desired dominant pole S_d

$S_d = -\zeta\omega_n + j\omega_n\sqrt{1-\zeta^2}$

01 k find

$$K_v = \lim_{s \to 0} \frac{sk}{s(s+2)(s+4)}$$

$$20 = \frac{k}{8}$$

$$\boxed{k = 160}$$

02 $T_s = \frac{4}{\zeta \omega_n}$

$\omega_n = 4$ rad/s

03 $\zeta = 0.5$

$$\boxed{s_d = -2 + j3.46}$$

Step 02: Angle of deficiency

$$\theta_i = \tan^{-1}[s_d - s_0]$$

Now $\theta_1 = \tan^{-1}\left[\frac{3.464}{2}\right] = 60°$

$\theta_2 = \tan^{-1}\left[\frac{3.464}{0}\right] = 90°$

$\theta_3 = \tan^{-1}\left[\frac{3.464}{3}\right] = 49.1°$

Pole angles $= 60° + 90° + 49.1° = 199.1°$

$\Rightarrow \angle G(s_d) = \Sigma z_\theta - \Sigma p_\theta$

$= 0 - 199.1$

$= -199.1°$

$\Rightarrow 180 - 199.1 = -19.1°$

$360 + \theta$ or $180 + \theta$ } to make it positive

Step 03: Lag Compensator

$$G_c(s) = K_c \frac{s+z}{s+p} \quad p < z$$

assume $\boxed{z = 0.5}$

$\theta_z = \tan^{-1}[-2 + j3.46 + 0.5]$

$= \tan^{-1}\left[-\frac{3.46}{1.5}\right]$

$= 180° - 66.562$

$\theta_z = 113.437°$

Now $\tan(\theta_p) = \frac{3.46}{p-2}$

$-2.306p + 4.61 = 3.46$

$p = 0.4887$

$$\boxed{p = 0.4887}$$

$$G_c(s) = \frac{K_c[s+0.5]}{[s+0.4887]}$$

Step 04: Compensated Equation

$$G_n(s) = G_p(s)\,G_c(s)$$

$$= \frac{k_c\,[s+0.5]}{s\,[s+2]\,[s+4]}$$

Step 05: Find k_c $\quad |G_n(s_d)| = 1 \Rightarrow \dfrac{|k_c|\,[s+0.5]}{|s|\,|s+2|\,|s+4|} = 1$

Part D: Lag Compensator Bode.

Given $= \dfrac{2}{[s]\,[s+2]\,[s+5]}$

$PM_{req} = 45°$

Step 01: Magnitude and angle

$$\Rightarrow G(j\omega) = \frac{2}{j\omega\,(j\omega+2)\,(j\omega+5)}$$

$$\Rightarrow |G(j\omega)| = \frac{2}{\omega\sqrt{\omega^2+4}\,\sqrt{\omega^2+25}}$$

$$\Rightarrow \angle G(j\omega) = 0° - 90° - \tan^{-1}\left[\frac{\omega}{2}\right] - \tan^{-1}\left[\frac{\omega}{5}\right]$$

Step 02: Find ω_{gcn} ~~= 0.916~~

$$PM = 180 + \angle G(j\omega_{gcn}) + 10°$$

$$\Rightarrow \angle G(j\omega_{gcn}) = -180 + 45° + 10°$$

$$\Rightarrow -90° - \tan^{-1}\left[\frac{\omega_{gcn}}{2}\right] - \tan^{-1}\left[\frac{\omega_{gcn}}{5}\right] = -125°$$

$$\Rightarrow -\tan^{-1}\left[\frac{A}{2}\right] - \tan^{-1}\left[\frac{A}{5}\right] = -35°$$

By calci $\quad \omega_{gcn} = A = 0.9163093139$ rad/s.

Step 03: $|G(j\omega_{gcn})|$ • 0.9163 rad/s → ω_{gcn} substitute

$$\Rightarrow \frac{2}{0.916\sqrt{4+0.916^2}\sqrt{25+0.916^2}} = 0.982 = |G(j\omega_{gcn})|.$$

01 $|G(j\omega)| \rightarrow$ eq

$\angle G(j\omega)$ - angle

02 $PM = 180 + \angle G(j\omega_{gcn})$

03 $|G(j\omega_{gcn})| = \beta \quad \frac{1}{\beta}\angle 1$

04 $G_c(s) = \dfrac{1+sT}{1+\beta T}$

05 $\dfrac{1}{T} = \dfrac{\omega_{gcn}}{10}$

06 $P = \dfrac{1}{\beta T}$

Step 04: $\beta = |G(j\omega_{gcn})|$

$\beta = 0.982$

$\Rightarrow \frac{1}{\beta} = \frac{1}{0.982} = 1.0183 \not< 1$ So $\omega_{gcn} = 0.7$

$\beta = |G(j\omega_{gcn})|$ so $\omega_{gcn} = 0.7$ substitute in $|G(j\omega_{gcn})|$

$\beta = 1.335$

$\frac{1}{\beta} = \frac{1}{1.335} < \checkmark$

Step 05: $\frac{1}{T} = \frac{\omega_{gcn}}{10}$

$\frac{1}{T} = \frac{0.7}{10} = 0.07$

$\boxed{T = 14.2865}$

Step 06:

$\rho = \frac{1}{\beta T} = 0.0524$

$z = \frac{1}{T} = 0.07$

$\rho < z$

Step 07: $G_c(s) = \frac{1 + s[14.286]}{1 + s[14.286][1.335]}$

Q2: As discussed in class, one application oriented compensator design using Matlab & simulation work.

Uncompensated System

The transfer function of the uncompensated system is given by

$$G(s) = \frac{4}{s(s+2)}$$

Simulink Model

The uncompensated system is implemented in MATLAB Simulink using a unity negative-feedback configuration.

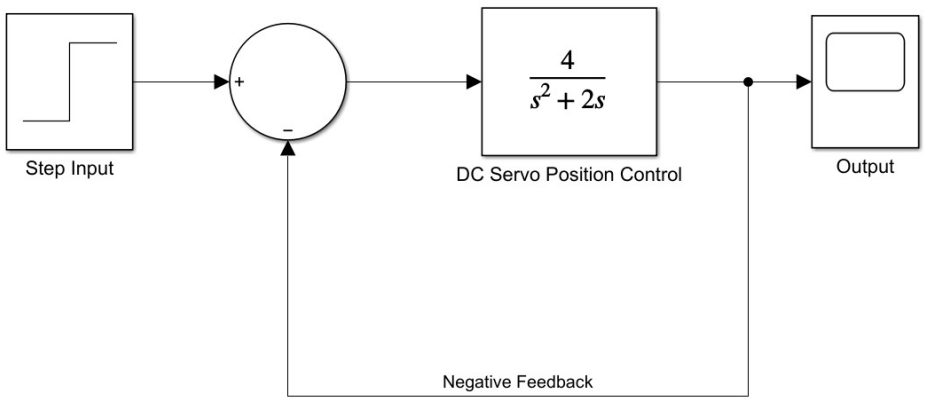


Figure 1: Simulink model of the uncompensated system.

Simulation Output

The simulation is performed using the uncompensated transfer function in Simulink.

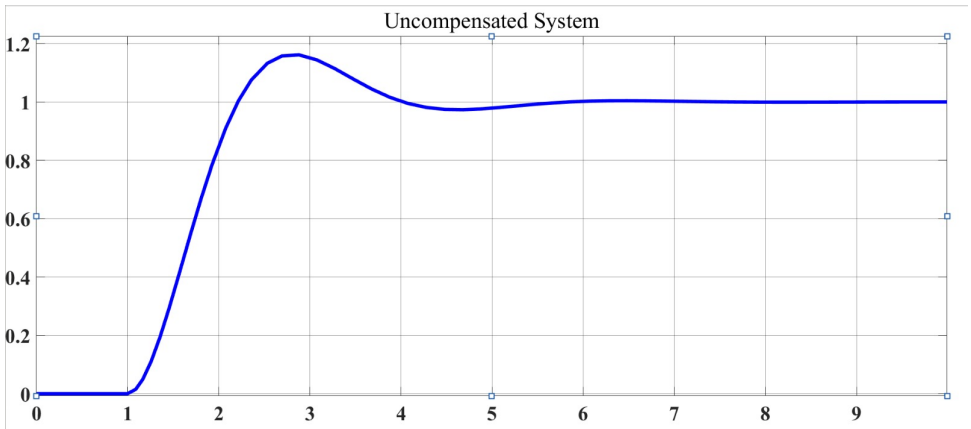


Figure 2: Simulation output of the uncompensated system.

MATLAB Code

The following MATLAB code is used to obtain the root locus and Bode plot of the uncompensated system.

```
clc;
clear;
close all;

s = tf('s');

% Uncompensated system
G = 4/(s*(s+2));

% Root locus
figure;
rlocus(G);
grid on;
title('Root Locus - Uncompensated System', 'FontSize', 14);
set(gca, 'FontSize', 13, 'LineWidth', 1.5);

% Make root locus lines thicker
h1 = findobj(gcf, 'Type', 'line');
set(h1, 'LineWidth', 2.5);
set(gcf, 'Color', 'white');

% Bode plot with stability margins
figure;
margin(G);
grid on;
title('Bode Plot - Uncompensated System', 'FontSize', 14);

% Improve Bode plot visibility
set(findall(gcf, 'Type', 'axes'), ...
    'FontSize', 13, 'LineWidth', 1.5);

h2 = findobj(gcf, 'Type', 'line');
set(h2, 'LineWidth', 2.5);
set(gcf, 'Color', 'white');
```

Root Locus

The root locus illustrates the movement of the closed-loop poles as the loop gain varies.

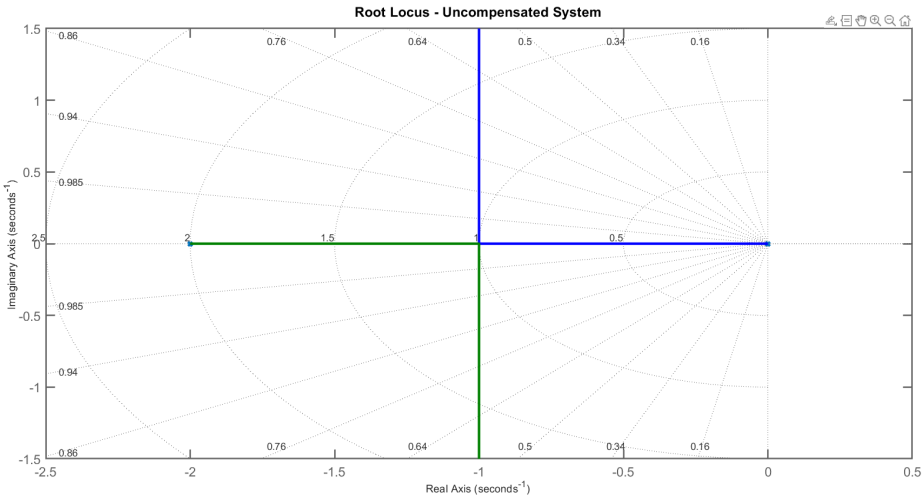


Figure 3: Root locus of the uncompensated system.

Bode Plot

The Bode plot represents the magnitude and phase response of the uncompensated open-loop system over a range of frequencies.

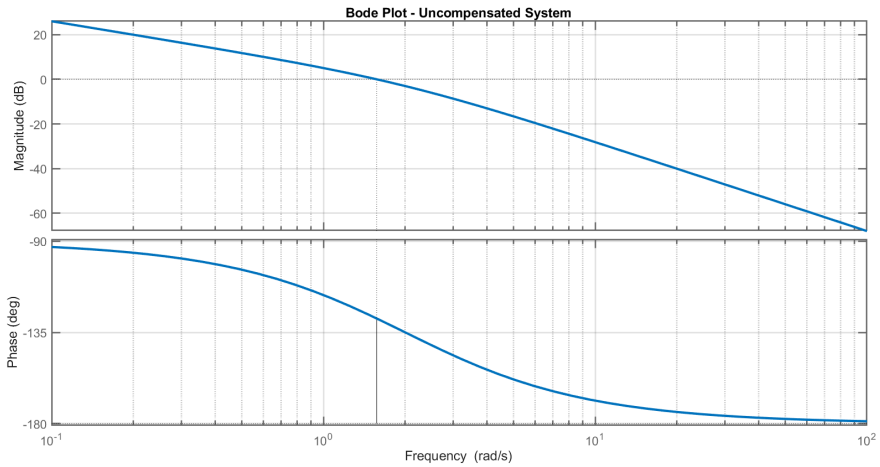


Figure 4: Bode plot of the uncompensated system.

Compensated System

The transfer function of the uncompensated plant is

$$G(s) = \frac{4}{s(s+2)}$$

The lead compensator is given by

$$C(s) = \frac{18.7s + 54.23}{s + 5.4}$$

The compensated open-loop transfer function is obtained by multiplying the lead compensator with the uncompensated plant:

$$G_c(s) = C(s)G(s)$$

$$G_c(s) = \frac{18.7s + 54.23}{s + 5.4} \cdot \frac{4}{s(s+2)}$$

Therefore,

$$G_c(s) = \frac{74.8s + 216.92}{s(s+2)(s+5.4)}$$

Simulink Model

The compensated system is implemented in MATLAB Simulink by connecting the lead compensator in series with the plant using a unity negative-feedback configuration.

The lead compensator Transfer Function block uses numerator `[18.7 54.23]` and denominator `[1 5.4]`. The plant Transfer Function block uses numerator `[4]` and denominator `[1 2 0]`.

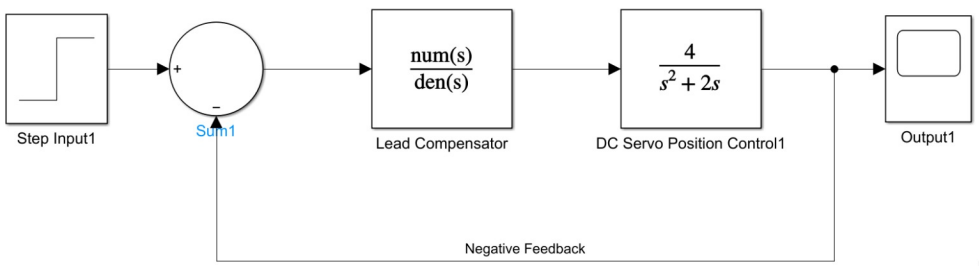


Figure 5: Simulink model of the compensated system.

Simulation Output

The compensated system is simulated using the lead compensator and plant connected in series within the unity feedback loop.

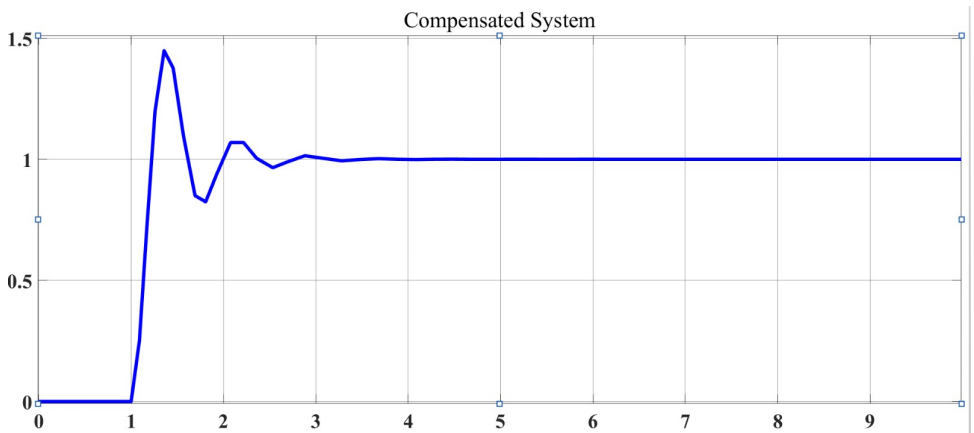


Figure 6: Simulation output of the compensated system.

MATLAB Code

The following MATLAB code is used to obtain the root locus and Bode plot of the compensated system.

Root Locus

The root locus illustrates the movement of the closed-loop poles as the loop gain varies for the compensated open-loop transfer function.

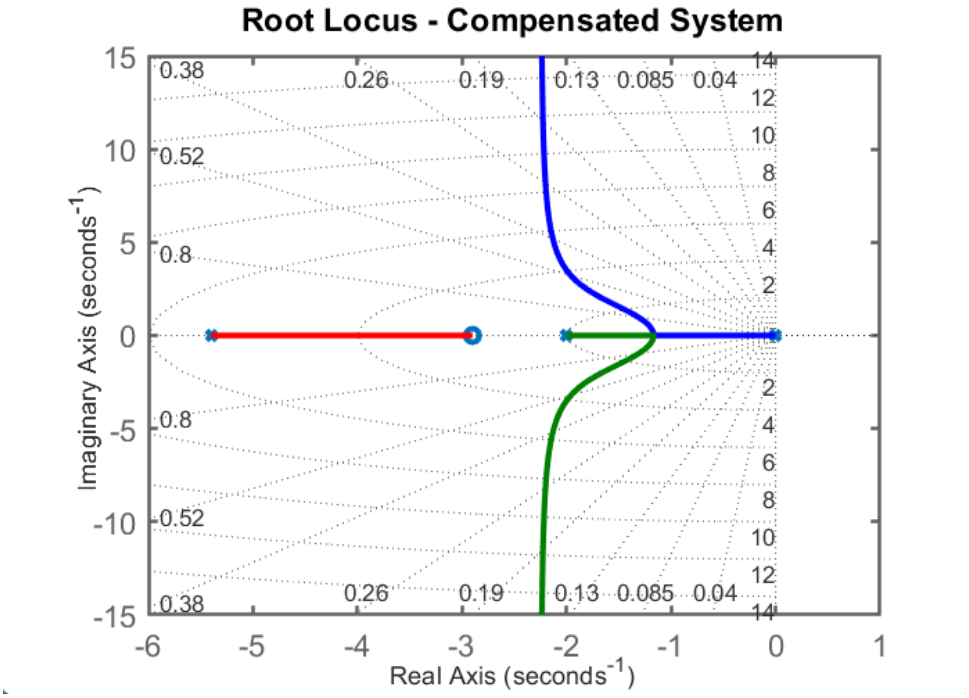


Figure 7: Root locus of the compensated system.

Bode Plot

The Bode plot represents the magnitude and phase response of the compensated open-loop system over a range of frequencies. The stability margins can be examined to evaluate the effect of the lead compensator.

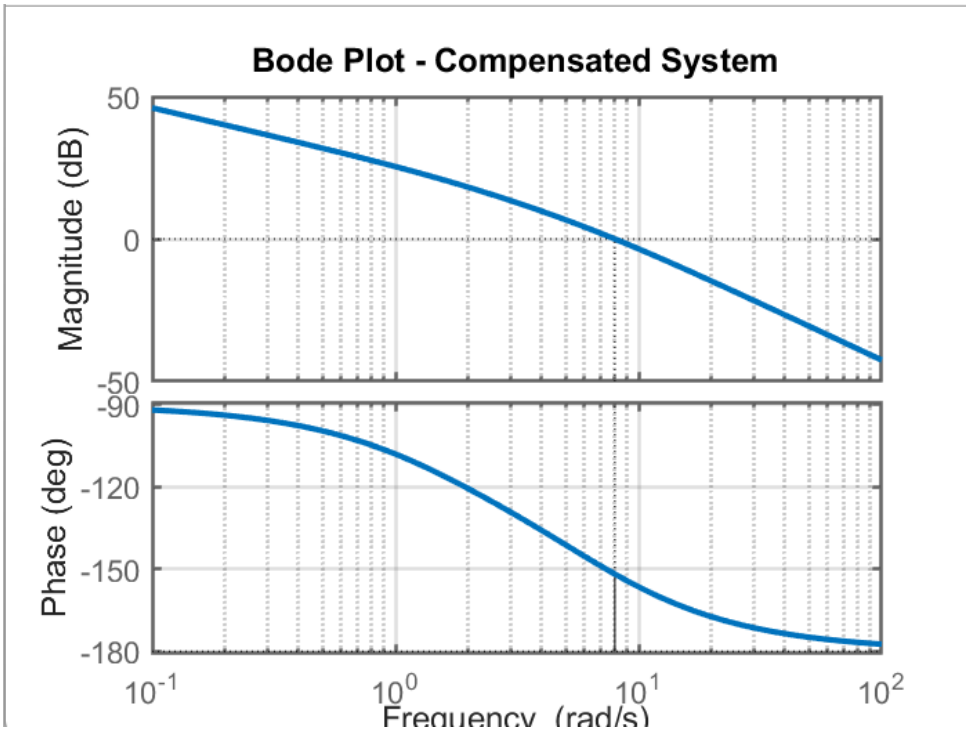


Figure 8: Bode plot of the compensated system.

Step Response and Comparison

Closed-Loop Transfer Functions

The uncompensated open-loop transfer function is

$$G(s) = \frac{4}{s(s+2)}$$

Assuming unity negative feedback, the closed-loop transfer function is

$$T_u(s) = \frac{G(s)}{1 + G(s)}$$

Substituting $G(s)$,

$$T_u(s) = \frac{4}{s^2 + 2s + 4}$$

The compensated open-loop transfer function is

$$G_c(s) = \frac{74.8s + 216.92}{s(s+2)(s+5.4)}$$

With unity negative feedback, the compensated closed-loop transfer function is

$$T_c(s) = \frac{G_c(s)}{1 + G_c(s)}$$

Therefore,

$$T_c(s) = \frac{74.8s + 216.92}{s(s+2)(s+5.4) + 74.8s + 216.92}$$

Expanding the denominator gives

$$T_c(s) = \frac{74.8s + 216.92}{s^3 + 7.4s^2 + 85.6s + 216.92}$$

MATLAB Simulation

The following MATLAB code is used to obtain and compare the unit step responses of both closed-loop systems.

```
clc;
clear;
close all;

s = tf('s');

% Uncompensated plant
G = 4/(s*(s+2));

% Lead compensator
C = (18.7*s + 54.23)/(s+5.4);

% Compensated open-loop system
Gc = C*G;

% Root locus
figure;
rlocus(Gc);
grid on;
title('Root Locus - Compensated System', 'FontSize', 14);

set(gca, 'FontSize', 13, 'LineWidth', 1.5);

% Increase root locus line thickness
h1 = findobj(gcf, 'Type', 'line');
set(h1, 'LineWidth', 2.5);

set(gcf, 'Color', 'white');

% Bode plot with stability margins
figure;
margin(Gc);
grid on;
title('Bode Plot - Compensated System', 'FontSize', 14);

% Improve Bode plot visibility
set(findall(gcf, 'Type', 'axes'), ...
    'FontSize', 13, 'LineWidth', 1.5);

% Increase Bode plot line thickness
h2 = findobj(gcf, 'Type', 'line');
set(h2, 'LineWidth', 2.5);

set(gcf, 'Color', 'white');
```

Simulink Model

The step response comparison can also be performed using Simulink. Two parallel paths are constructed, one for the uncompensated system and the other for the compensated system. Both paths receive the same unit step input, and their outputs are connected to a common Scope for comparison.

Simulation Results

The unit step responses obtained from MATLAB are shown below.

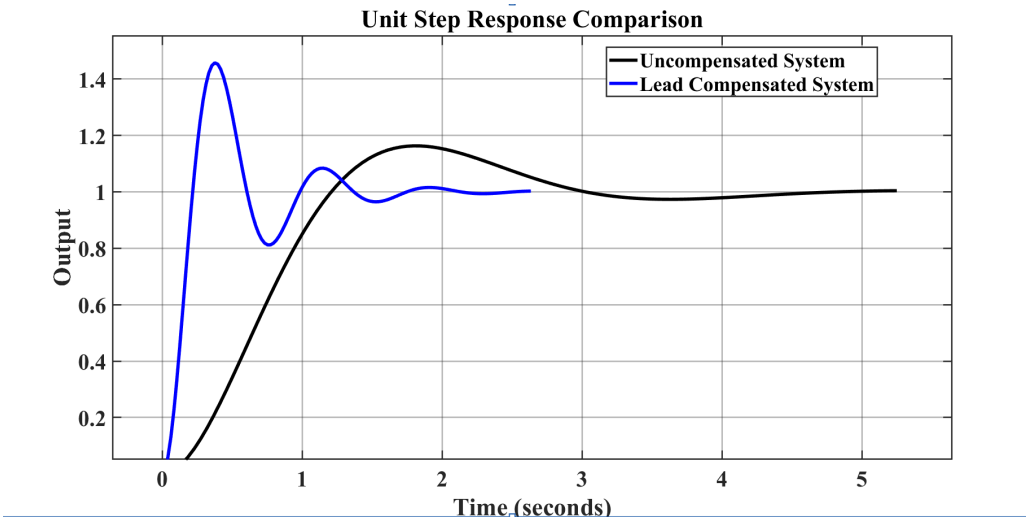


Figure 9: Comparison of uncompensated and compensated step responses

Q3: Please find the attached file and implement this paper on Matlab. & simulation. Compare results.

Paper Title: A Study of state feedback controllers for pole placements.

Equations & Derivations:

⇒ Equation 01: $T_m = K_t i_a$

⇒ Equation 02: $V_m(t) = L_a \frac{di_a}{dt} + R_a i_a + e_b$

⇒ Equation 03: $K_t i_a = J_m \frac{d\omega_m}{dt} + b_m \frac{d\theta_m}{dt}$

⇒ Equation 04: $\frac{\theta_m}{V_{in}(s)} = \frac{K_t}{s[(L_a s + R_a)(J_m s + b_m) + K_t K_b)]}$

⇒ Equation 05: $\frac{\theta_m(s)}{V_{in}(s)} = \frac{k / R_a J_m}{s^2 + s\left[\frac{R_a b_m + k^2}{R_a J_m}\right]}$

⇒ Equation 06: $\theta_m'' = -\left[\frac{R_a b_m + k^2}{R_a J_m}\right]\theta_m' + \frac{k}{R_a J_m} V_{in}$

⇒ Equation 07: $\begin{bmatrix} x_1' \\ x_2' \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & -\left[\frac{R_a b_m + k^2}{R_a J_m}\right] \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{k}{R_a J_m} \end{bmatrix} V_{in}$

⇒ Equation 08: $y = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$

⇒ Equation 09: $x'(t) = Ax(t) + Bu(t)$

⇒ Equation 10: $y(t) = Cx(t) + Du(t)$.

Equation 11: $AN_x + BN_u = 0$

Equation 12: $CN_x + DN_u = I$

Equation 13: $u = -Kx + Nr$

Equation 14: $\dot{x}_N = r - Cx$

Equation 15: $\dot{x} = Ax + Bu$

Equation 16: $y = Cx$

Equation 17:
$$\begin{bmatrix} \dot{x} \\ \dot{x}_N \end{bmatrix} = \begin{bmatrix} A & 0 \\ -C & 0 \end{bmatrix} \begin{bmatrix} x \\ x_N \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u + \begin{bmatrix} 0 \\ I \end{bmatrix} r$$

Equation 18:
$$y = \begin{bmatrix} C & 0 \end{bmatrix} \begin{bmatrix} x \\ x_N \end{bmatrix}$$

Equation 19:
$$\begin{bmatrix} \dot{x} \\ \dot{x}_N \end{bmatrix} = \begin{bmatrix} A - BK & BKe \\ -C & 0 \end{bmatrix} \begin{bmatrix} x \\ x_N \end{bmatrix} + \begin{bmatrix} 0 \\ I \end{bmatrix} r$$

Equation 20:
$$y = \begin{bmatrix} C & 0 \end{bmatrix} \begin{bmatrix} x \\ x_N \end{bmatrix}$$

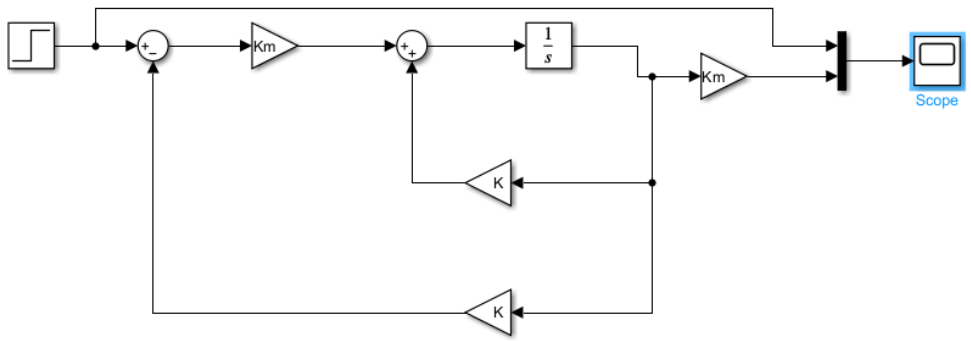


Figure 2: Simulink Model of Full State Feedback Control (SFB).

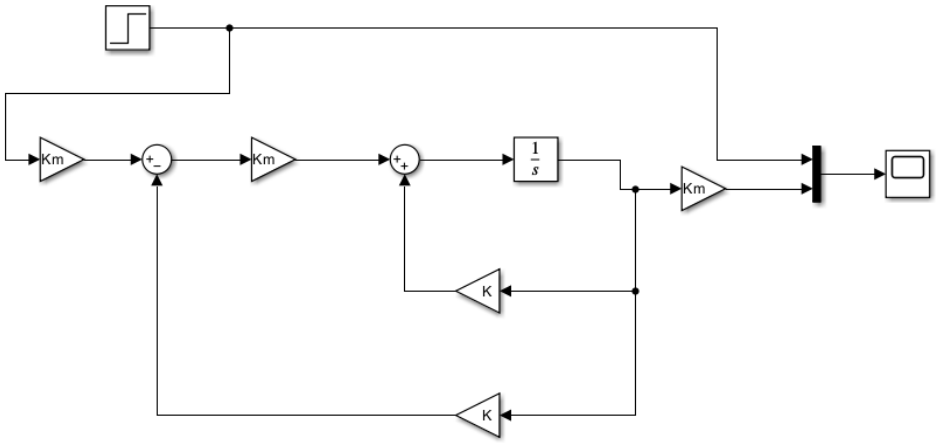


Figure 3: Simulink Model of State Feedback Control with Feed-Forward Gain.

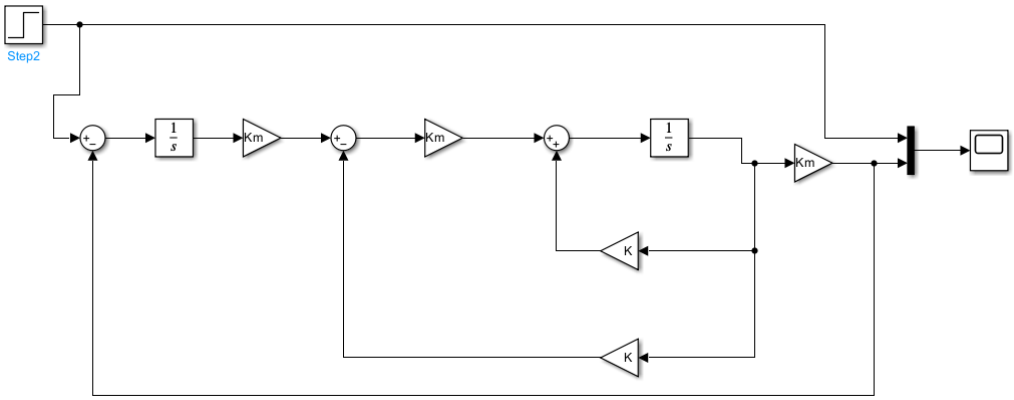


Figure 4: Simulink Model of Integral Control with State Feedback.

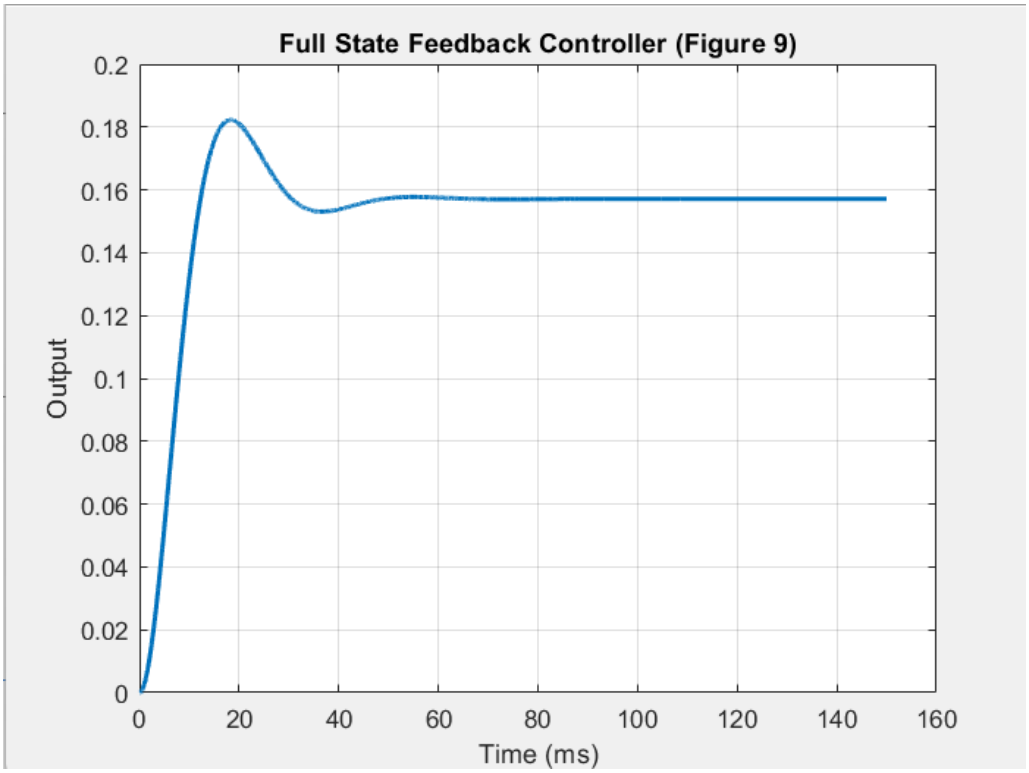


Figure 5: Simulation Result of Full State Feedback Control.

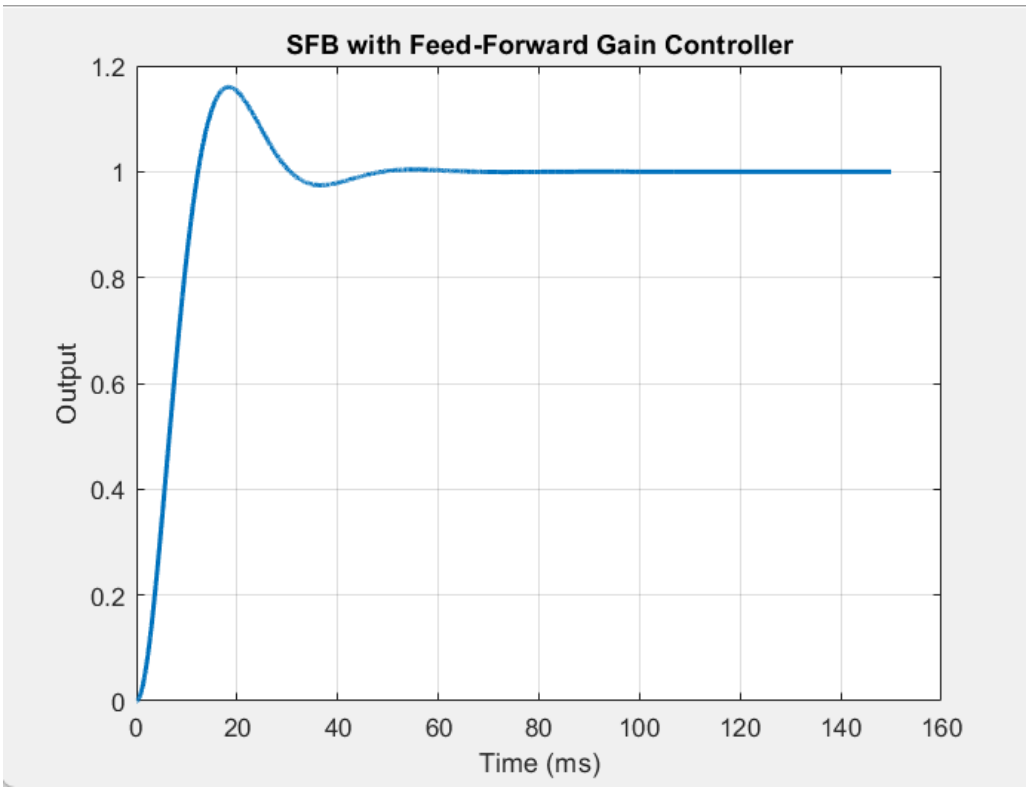


Figure 6: Simulation Result of State Feedback Control with Feed-Forward Gain.

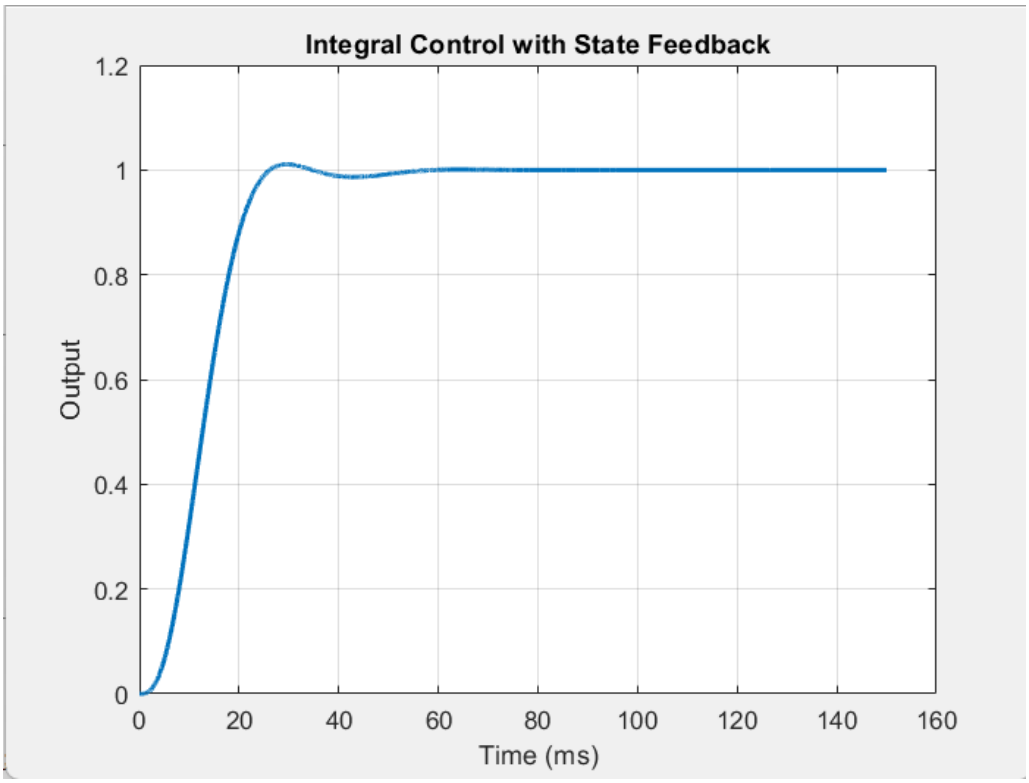


Figure 7: Simulation Result of Integral Control with State Feedback.

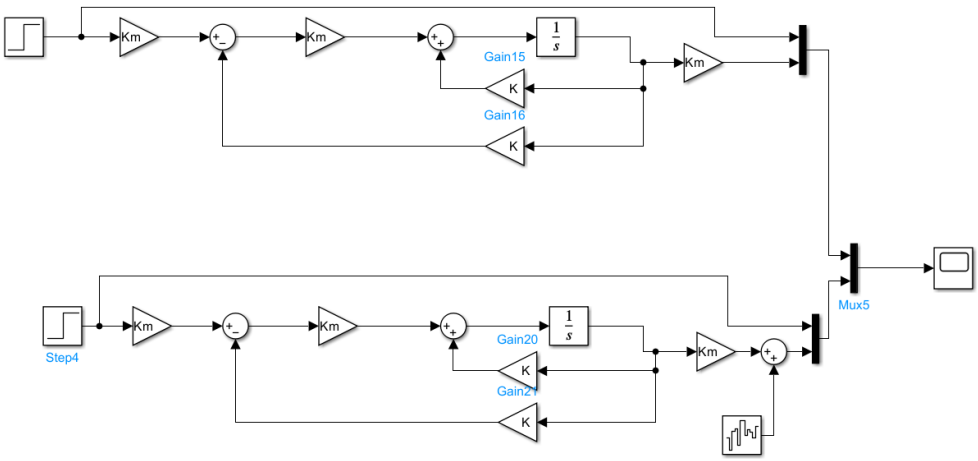


Figure 8: State Feedback Control with Feed-Forward Controller under Induced Noise.

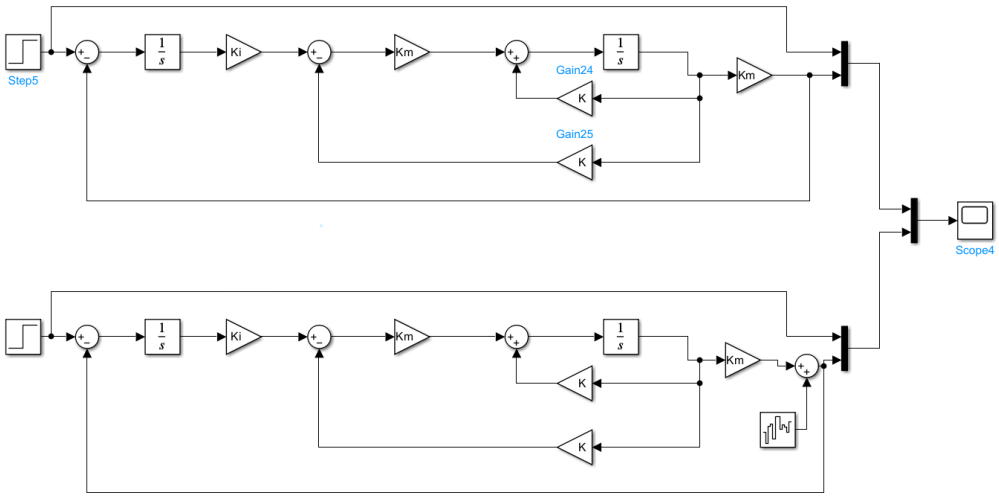


Figure 9: Integral Control with State Feedback under Induced Noise.

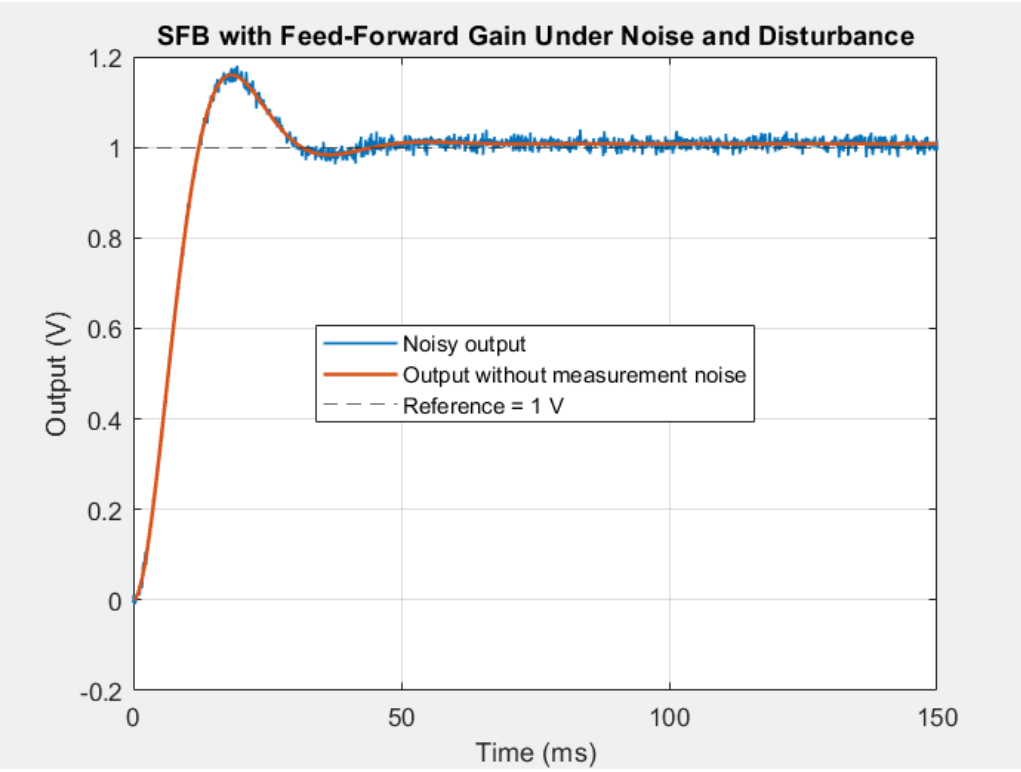


Figure 10: Simulation Result of State Feedback with Feed-Forward Controller under Induced Noise.

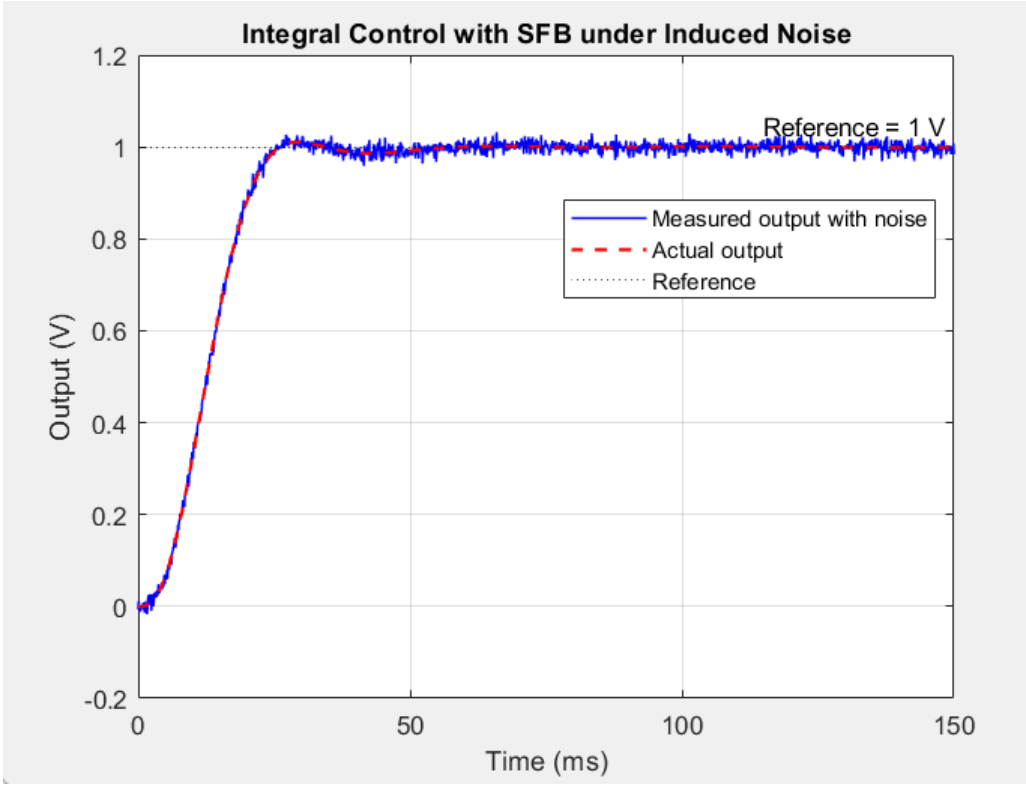


Figure 11: Simulation Result of Integral Control with State Feedback under Induced Noise.

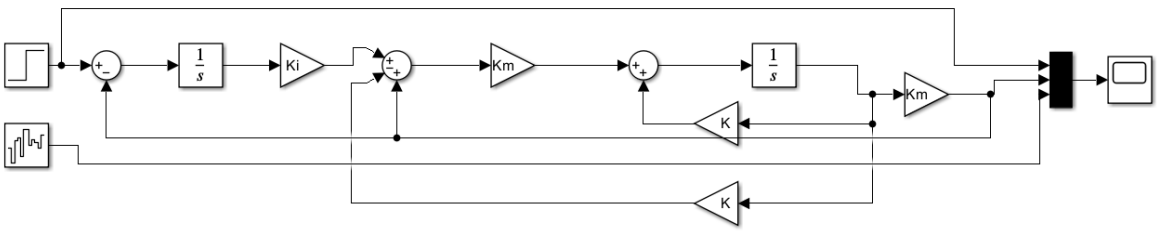


Figure 12: Integral State Feedback Control with Noise at the Input.

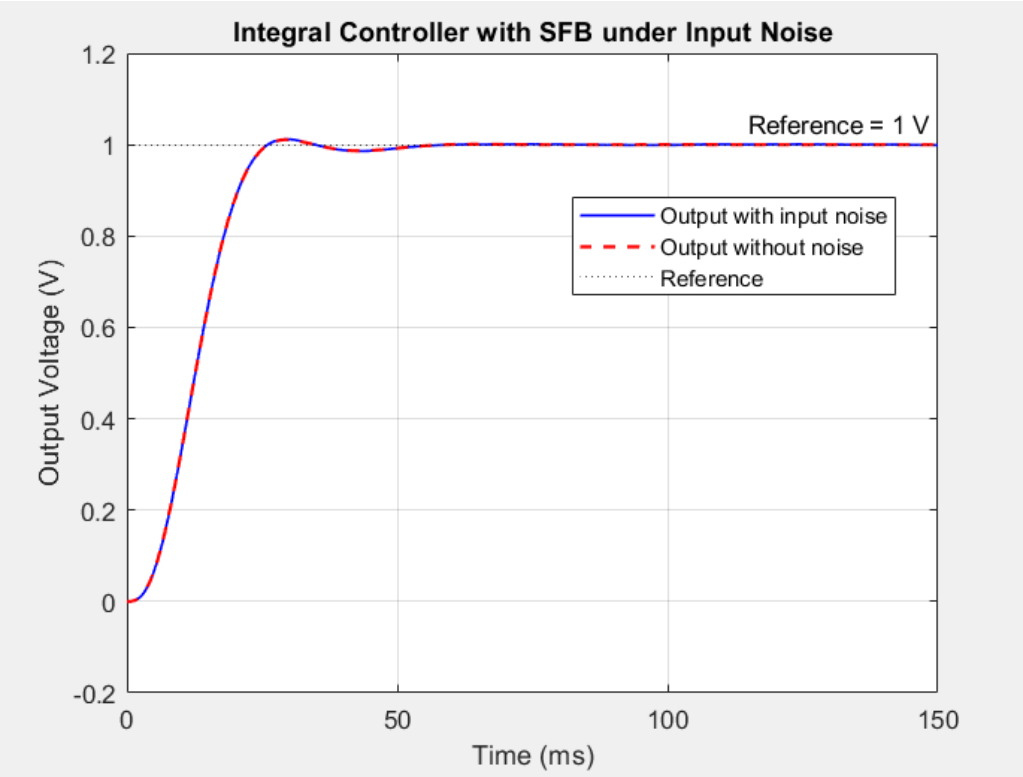


Figure 13: Simulation Result of Integral Control with State Feedback under Input Noise.

Q3: MATLAB CODE

```
clc;
clear;
close all;

%% DC Servo Motor Parameters (from the paper)
J  = 1.2e-6;
b  = 6.71e-6;
Km = 0.015;
R  = 2.02;

%% State-Space Model
A = [0 1;
     0 -(R*b + Km^2)/(R*J)];

B = [0;
     Km/(R*J)];

C = [1 0];
D = 0;

%% Desired Specifications
Ts = 0.040;
OS = 16;

zeta = -log(OS/100) / ...
       sqrt(pi^2 + (log(OS/100))^2);

wn = 4/(zeta*Ts);

%% Desired Poles
p1 = -zeta*wn + 1i*wn*sqrt(1-zeta^2);
p2 = -zeta*wn - 1i*wn*sqrt(1-zeta^2);
p = [p1 p2];

%% State Feedback Gain
K = place(A,B,p);

%% Full State Feedback Controller
Acl = A - B*K;

%% Closed-loop response
sys_sfb = ss(Acl,B,C,D);
t = 0:1e-5:0.15;
r = ones(size(t));
[y,t] = lsim(sys_sfb,r,t);

%% Plot response
figure;
plot(t*1000,y,'LineWidth',1.8);
grid on;
xlabel('Time (ms)');
ylabel('Output');
title('Full State Feedback Controller (Figure 9)');

%% Performance values
info = stepinfo(y,t,0.157);
```

```
57
58 fprintf('K1 = %.6f\n',K(1));
59 fprintf('K2 = %.8f\n',K(2));
60 fprintf('Final output = %.6f\n',y(end));
61 fprintf('Settling time = %.3f ms\n', ...
62         info.SettlingTime*1000);
63 fprintf('Overshoot = %.2f %%\n',info.Overshoot);
```

Listing 1: MATLAB code for the full state feedback controller (Figure 09).

```
1  clc;
2  clear;
3  close all;
4
5  J  = 1.2e-6;
6  b  = 6.71e-6;
7  Km = 0.015;
8  R  = 2.02;
9
10 A = [0 1;
11      0 -(R*b + Km^2)/(R*J)];
12 B = [0; Km/(R*J)];
13 C = [1 0];
14
15 Ts = 0.040;
16 OS = 16;
17
18 zeta = -log(OS/100) / ...
19        sqrt(pi^2 + (log(OS/100))^2);
20 wn = 4/(zeta*Ts);
21
22 p = [-zeta*wn + 1i*wn*sqrt(1-zeta^2), ...
23      -zeta*wn - 1i*wn*sqrt(1-zeta^2)];
24
25 K = place(A,B,p);
26 Acl = A - B*K;
27
28 Kr = -1/(C*(Acl\B));
29
30 sys_ff = ss(Acl,B*Kr,C,0);
31
32 t = 0:1e-5:0.15;
33 r = ones(size(t));
34 [y,t] = lsim(sys_ff,r,t);
35
36 figure;
37 plot(t*1000,y,'LineWidth',1.8);
38 grid on;
39 xlabel('Time (ms)');
40 ylabel('Output');
41 title('SFB with Feed-Forward Gain Controller');
42
43 info = stepinfo(y,t,1);
44
45 fprintf('State feedback gain K =\n');
46 disp(K);
47 fprintf('Feed-forward gain Kr = %.6f\n',Kr);
48 fprintf('Final output = %.6f V\n',y(end));
```

```
fprintf('Steady-state error = %.6f V\n',abs(1-y(end)));
fprintf('Settling time = %.3f ms\n',info.SettlingTime*1000);
fprintf('Overshoot = %.2f %%\n',info.Overshoot);
```

Listing 2: MATLAB code for state feedback with feed-forward gain (Figure 10).

```
clc;
clear;
close all;

J  = 1.2e-6;
b  = 6.71e-6;
Km = 0.015;
R  = 2.02;

A = [0 1;
     0 -(R*b + Km^2)/(R*J)];
B = [0; Km/(R*J)];
C = [1 0];

Ts = 0.040;
OS = 16;

zeta = -log(OS/100) / ...
       sqrt(pi^2 + (log(OS/100))^2);
wn = 4/(zeta*Ts);

p = [-zeta*wn + 1i*wn*sqrt(1-zeta^2), ...
     -zeta*wn - 1i*wn*sqrt(1-zeta^2)];

Aaug = [A, zeros(2,1);
       -C, 0];
Baug = [B; 0];

% Illustrative third pole, not specified in the paper
p3 = -5/Ts;
paug = [p, p3];

Kaug = place(Aaug,Baug,paug);
Kx = Kaug(1:2);
Ki = Kaug(3);

Acl = Aaug - Baug*Kaug;
Bref = [0; 0; 1];
Cout = [C 0];

sys_integral = ss(Acl,Bref,Cout,0);

t = 0:1e-5:0.15;
r = ones(size(t));
[y,t] = lsim(sys_integral,r,t);

figure;
plot(t*1000,y,'LineWidth',1.8);
grid on;
xlabel('Time (ms)');
ylabel('Output');
title('Integral Control with State Feedback');
```

```
53
54 info = stepinfo(y,t,1);
55
56 fprintf('Final output = %.4f V\n',y(end));
57 fprintf('Steady-state error = %.4f V\n',abs(1-y(end)));
58 fprintf('Settling time = %.2f ms\n',info.SettlingTime*1000);
59 fprintf('Overshoot = %.2f %%\n',info.Overshoot);
```

Listing 3: MATLAB code for integral state feedback control (Figure 11).

```
1  clc;
2  clear;
3  close all;
4
5  J  = 1.2e-6;
6  b  = 6.71e-6;
7  Km = 0.015;
8  R  = 2.02;
9
10 A = [0 1;
11      0 -(R*b + Km^2)/(R*J)];
12 B = [0; Km/(R*J)];
13 C = [1 0];
14
15 Ts = 0.040;
16 OS = 16;
17
18 zeta = -log(OS/100) / ...
19        sqrt(pi^2 + (log(OS/100))^2);
20 wn = 4/(zeta*Ts);
21
22 p = [-zeta*wn + 1i*wn*sqrt(1-zeta^2), ...
23      -zeta*wn - 1i*wn*sqrt(1-zeta^2)];
24
25 K = place(A,B,p);
26 Acl = A - B*K;
27
28 Kr = -1/(C*(Acl\B));
29 sys_ff = ss(Acl,B*Kr,C,0);
30
31 dt = 1e-4;
32 t = (0:dt:0.15)';
33 r = ones(size(t));
34
35 % Illustrative noise settings, not specified by the paper
36 rng(1);
37 noise_std = 0.01;
38 noise = noise_std*randn(size(t));
39
40 disturbance = zeros(size(t));
41 disturbance(t >= 0.02) = 0.05;
42
43 u_total = Kr*r + disturbance;
44 sys_dist = ss(Acl,B,C,0);
45
46 [y_dist,tout] = lsim(sys_dist,u_total,t);
47 y_noisy = y_dist + noise;
48
```

```
figure;
plot(tout*1000,y_noisy,'LineWidth',1);
hold on;
plot(tout*1000,y_dist,'LineWidth',1.5);
yline(1,'--');
grid on;

xlabel('Time (ms)');
ylabel('Output (V)');
title('SFB with Feed-Forward Gain Under Noise and Disturbance');
legend('Noisy output','Output without measurement noise', ...
       'Reference = 1 V','Location','best');

fprintf('Paper reference output = 0.693 V\n');
fprintf('Paper reference SSE = 0.306 V\n');
fprintf('Paper reference settling time = 40.5 ms\n');
fprintf('Paper reference overshoot = 16 %%\n');
```

Listing 4: MATLAB code for feed-forward state feedback under noise and disturbance (Figure 14).

```
clc;
clear;
close all;

J  = 1.2e-6;
b  = 6.71e-6;
Km = 0.015;
R  = 2.02;

A = [0 1;
     0 -(R*b + Km^2)/(R*J)];
B = [0; Km/(R*J)];
C = [1 0];
D = 0;

Ts = 0.040;
OS = 16;

zeta = -log(OS/100)/sqrt(pi^2 + log(OS/100)^2);
wn = 4/(zeta*Ts);

p = [-zeta*wn + 1i*wn*sqrt(1-zeta^2), ...
     -zeta*wn - 1i*wn*sqrt(1-zeta^2)];

Aaug = [A, zeros(2,1);
        -C, 0];
Baug = [B; 0];

% Illustrative third pole, not specified in the paper
p3 = -5/Ts;
Kaug = place(Aaug,Baug,[p p3]);

Kx = Kaug(1:2);
Ki = Kaug(3);

Acl = [A-B*Kx, -B*Ki;
       -C,       0];
```

```
38
39 % Inputs: reference and measurement noise
40 Bcl = [0 0;
41        0 0;
42        1 -1];
43
44 Ccl = [C 0];
45 Dcl = [0 0];
46
47 sys_int = ss(Acl,Bcl,Ccl,Dcl);
48
49 dt = 1e-4;
50 t = (0:dt:0.15)';
51 reference = ones(size(t));
52
53 % Illustrative measurement noise
54 rng(1);
55 noise_std = 0.01;
56 noise = noise_std*randn(size(t));
57
58 U = [reference, noise];
59 [y_true,tout] = lsim(sys_int,U,t);
60 y_measured = y_true + noise;
61
62 figure;
63 plot(tout*1000,y_measured,'b','LineWidth',0.9);
64 hold on;
65 plot(tout*1000,y_true,'r--','LineWidth',1.3);
66 yline(1,'k:','Reference = 1 V');
67
68 grid on;
69 xlabel('Time (ms)');
70 ylabel('Output (V)');
71 title('Integral Control with SFB under Induced Noise');
72 legend('Measured output with noise','Actual output', ...
73        'Reference','Location','best');
```

Listing 5: MATLAB code for integral state feedback under induced noise (Figure 15).